

Fermat's Attack on RSA

Diya Jain, Meghma Sen and Sara Lorca Garrido

Department of Mathematics, University of South Florida

MAD 4472 : Modern Cryptography

Dr. Jean-Francois Biasse

November 27, 2024

Description

This project highlights the significance of RSA security and explores vulnerabilities associated with insufficiently secure RSA implementations. It focuses on leveraging Fermat's Factorization method to exploit these weaknesses. The goal is to decrypt a message using specific RSA parameters, employing Fermat's method to factorize the modulus, and capture the flag embedded in the ciphertext.

Theoretical Background: Fermat's Factoring Method

Fermat's Factorization is a classical method used to factorize integers, particularly effective when the number to be factored is a product of two prime numbers that are close to each other. The technique uses the difference of squares identity:

$$N = a^2 - b^2 = (a + b)(a - b)$$

Where:

- N is the number to be factored.
- a and b are integers such that $a^2 \geq N$ and b^2 is the largest perfect square less than or equal to $a^2 - N$.

The approach assumes that N can be expressed as the difference of two squares. By rearranging the equation, we have:

$$N = (a + b)(a - b)$$

From this, the two factors of N can be identified as $(a + b)(a - b)$

Algorithm

1. Compute $a = \lceil \sqrt{N} \rceil$,
2. Calculate $b^2 = a^2 - N$.
3. Check if b^2 is a perfect square. If it is, compute $b = \sqrt{b^2}$
4. Retrieve the factors of N as $(a + b)(a - b)$.
5. If b^2 is not a perfect square, increment a and repeat the process until factorization is found.

Application to RSA

In RSA encryption, the modulus N is the product of two large prime numbers p and q . Fermat's method becomes effective if p and q are close in value, as b will be small, making the factorization process computationally possible. If an attacker can successfully factor N into p and q , they can derive the private key d from the public key parameters and decrypt the ciphertext.

Course Relevance

As mentioned above this project is about Fermat's Attack on RSA, which is a crucial topic in cryptography as it highlights the importance of choosing secure RSA parameters. In this course it helps us understand the RSA one-way function, which is part of the Module 6 content.

"The RSA one-way function relies on Euler's theorem, which is a generalization of the Fermat's little theorem that for any prime p , and integer x such that $p \nmid x$, we have $x^{p-1} \equiv 1 \pmod{p}$." (add citation for class notes)

Fermat's Attack shows how RSA functions can be vulnerable if the difference between the two prime factors (p and q) is too small and relies on the importance of choosing appropriate values for our factors. Lastly, it also provides an insight into factorization methods, which are critical in RSA security.

Flag Value

The goal of this project was to recover the flag encrypted in the ciphertext, for which we applied a code in Maple to decrypt the message and we got the following flag value:

ABCTF{i_h4ve_an_RSA_fetish ;})

Maple Code

This code below implements **Fermat's factorization method**, an algorithm to find two factors of a number n by exploiting the difference of squares. It iteratively adjusts values to show factors efficiently.

```
# Perform Fermat's factorization to find the prime factors 'p' and 'q' of 'n'.
a := ceil(isqrt(n)); # Start from the smallest integer 'a' such that a^2 ≥ n.
iteration_count := 0; # Counter for the number of iterations.

while true do
    iteration_count := iteration_count + 1;
    b_squared := a^2 - n; # Calculate the difference b^2 = a^2 - n.
    b := sqrt(b_squared); # Check if 'b_squared' is a perfect square.

    if b = floor(b) then # If 'b' is an integer, we found a solution.
        p := a - b; # One factor is 'p = a - b'.
        q := a + b; # The other factor is 'q = a + b'.

        if p * q = n then # Verify the factors multiply to 'n'.
            print("Found factors after ", iteration_count, " iterations.");
            print(" p = ", p);
            print(" q = ", q);
            break; # Exit the loop when factors are found
        end if;
    end if;
    a := a + 1; # Increment 'a' and repeat the process.
end do;
```

This procedure implements **modular exponentiation**, an efficient algorithm to compute $base^{exp} \pmod{modulus}$ using the method of exponentiation by squaring, optimized for large numbers.

```

# Defined a modular exponentiation function to decrypt the ciphertext.
ModularExponentiation := proc( base, exponent, modulus)
    local result, i, exp_bits;

    result := 1; # Initialize the result as 1.
    exp_bits := [seq(iquo(exponent, 2^i) mod 2, i = 0 .. floor(log[2](exponent)))]; # Com

    for i from nops(exp_bits) - 1 to 0 by -1 do # Loop through bits from MSB to LSB.
        result := (result^2) mod modulus; # Square the result modulo `modulus`
        if exp_bits[i + 1] = 1 then
            result := (result * base) mod modulus; # Multiply if the current bit is 1.
        end if;
    end do;
    return result; # Return the computed result.
end proc;

```

This procedure converts a numeric value into its corresponding text representation by interpreting its binary form as ASCII character codes.

```

# Convert the plaintext number `m` into a readable string.
convertToText := proc( num)
    local result, binaryStr, i, byteStr, charCode, charList;
    charList := [ ]; # Initialize an empty list to store characters.

    binaryStr := cat(convert(num, binary)); # Convert `m` to a binary string.
    print("Binary string:", binaryStr);

    binaryStr := StringTools:-PadLeft(binaryStr, 8 * ceil( (1/8) * length(binaryStr)), "0"); # Pad the binary string
    print("Padded binary string:", binaryStr);

    for i from 1 by 8 to length(binaryStr) do # Process the binary string in 8-bit chunks.
        byteStr := substring(binaryStr, i .. i + 7); # Extract 8 bits.
        charCode := convert(byteStr, decimal, 2); # Convert 8 bits to decimal.
        charList := [op(charList), sprintf("%c", charCode)]; # Append corresponding character to the list.
        #print("charList:", charList);
    end do;

    result := cat(charList[ ]); # Concatenate all characters into a single string.
    # print("Final result:", result);
    return result;
end proc;

```

Functionalities of Code

This code performs the decryption of an RSA-encrypted message and recovers the plaintext flag. It integrates various algorithms and methods, ensuring all critical components are addressed for successful decryption. Below are the detailed steps and their importance:

1. Factorization with Fermat's Factorization Method

- **Purpose:** This step factors the RSA modulus n into its prime factors p and q , which are essential for computing the private key.
- **Process:**
 1. The method starts with $a = \lceil \sqrt{N} \rceil$ and iteratively calculates $b^2 = a^2 - N$.
 2. When b is an integer, $p = a - b$ and $q = a + b$ is determined as the prime factors.
- **Importance:** Breaking n into p and q allows computation of $\phi(n)$, the totient required for decryption.

2. Compute $\phi(n)$

- **Purpose:** $\phi(n) = (p-1)(q-1)$ is used to calculate the private decryption exponent d .

- **Importance:** $\phi(n)$ is a crucial part of the RSA algorithm as it defines the modular multiplicative inverse relationship between e (encryption exponent) and d (decryption exponent).

3. Calculate the Private Key d

- **Purpose:** Using e , the public encryption exponent, and $\phi(n)$, the private key d is computed as $d \equiv e^{-1} \bmod \phi(n)$.
- **Importance:** d is required to decrypt the ciphertext c into the plaintext message m .

4. Efficient Modular Exponentiation

- **Method:** The `ModularExponentiation` function performs fast computation of $m \equiv c^d \bmod n$ using exponentiation by squaring.
- **Importance:** This step efficiently decrypts the ciphertext, even for exceptionally large values of c , d , and n , ensuring practicality in cryptographic computations.

5. Convert Numeric Message to Text

- **Method:** The `convertToText` function translates the numeric plaintext m into a human-readable ASCII string:
 - Converts m to binary.
 - Pads the binary string to ensure proper 8-bit alignment.
 - Interprets every 8-bit segment as an ASCII character.
- **Importance:** Converts the decrypted numeric message into a meaningful flag or message.

6. Recover the Flag

- **Final Output:** The decrypted message, now in readable format, is printed as the "flag," showing successful decryption.
- **Importance:** Completes the decryption process, making the flag accessible for further use.

Significance of the Code

- This program combines cryptographic theory with computational techniques:
- It proves the vulnerability of RSA when the modulus n is factorable.
- Efficiently manages large numbers using Fermat's method, modular arithmetic, and binary processing.
- Provides a step-by-step approach from factorization to decryption and plaintext recovery, essential in cryptographic analysis and security testing.